

Notas

Unidad 1: Ingeniería de Software en Contexto

La ingeniería de software es una disciplina de ingeniería que se encarga de todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

La ingeniería de Software tiene un alcance transversal en las organizaciones:

- Disciplinas técnicas:
 - Testing
 - Despliegue
- Disciplinas de Gestión:
 - Planificación de proyecto
 - Monitoreo y control de proyectos
- Disciplinas de Soporte:
 - Gestión de Configuración de Software
 - Aseguramiento de Calidad
 - Métricas

Lo más importante para un buen desarrollo de software es:

1. Involucramiento del usuario
2. Apoyo de la gerencia
3. Enunciado claro de los requerimientos

Ciclos de Vida

Serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo. Definen la forma en que se estructura y organiza el desarrollo de un sistema, marcando los pasos que se atraviesan desde su inicio hasta su finalización. Son modelos teóricos o metodologías que guían cómo se debería desarrollar un software.

Pueden ser:

- Secuencial: Sigue un enfoque lineal y ordenado. Cada etapa se debe finalizar por completo antes de pasar a la siguiente. Su principal limitación es la falta de flexibilidad, ya que los cambios en etapas avanzadas pueden ser costosos y difíciles de implementar.
- Iterativo/Incremental: Divide el desarrollo en unidades de tiempo denominadas iteraciones, y al finalizar cada una se entrega un incremento o parte funcional del sistema. El incremento es pequeño, pero funcional. Permite recibir retroalimentación temprana del cliente y adaptarse a cambios de requisitos.
- Recursivo: En cada iteración, se va a obtener un incremento, pero como versión incompleta, es decir que no es funcional, hasta que se llegue a la versión final.

Tanto los productos como los proyectos, tienen ciclos de vida y antes de iniciarlos es necesario determinar cuál es el que van a aplicar.

El ciclo de vida en los proyectos de software es una representación de un proceso. Grafica una descripción del proceso desde una perspectiva particular. Los modelos especifican las fases del proceso y el orden en el cual se llevan a cabo.

Procesos de Software

"Conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados"

Incluye cuatro actividades fundamentales comunes a todos los procesos de software:

1. Especificación del software, donde clientes e ingenieros definen el software que se producirá y las restricciones en su operación.
2. Desarrollo del software, donde se diseña y programa el software
3. Validación del software, donde se verifica el software para asegurar que sea lo que el cliente requiere.
4. Evolución del software, donde se modifica el software para reflejar los requerimientos cambiantes del cliente y del mercado.

Un proceso de desarrollo se puede aplicar a varios ciclos de vida. La elección del ciclo de vida depende de la estrategia y este debe permitir seleccionar los artefactos a producir, definir actividades y roles, y modelar conceptos.

Existen tres factores determinantes del proceso:

- Procedimientos y Métodos.
- Personas motivadas, capacitadas y con habilidades.
- Herramientas y Equipos

Los procesos pueden ser definidos o empíricos

Empírico

Asume procesos complicados con variables cambiantes por lo que no siempre se obtiene el mismo resultado. La administración y control es a través de inspecciones frecuentes y adaptaciones.

Sus pilares son la inspección, adaptación y transparencia de la información ante todos los integrantes del equipo de trabajo.

- Transparencia: garantiza que los aspectos significativos del proceso y del trabajo sean visibles para quienes son responsables del resultado. No se trata solo de "ver" en qué se está trabajando, sino de tener un entendimiento compartido.

- Inspección: el equipo debe revisar regularmente sus artefactos (el producto, el código, el backlog) y su progreso hacia los objetivos para detectar variaciones indeseadas o problemas.

o Detección temprana.

o Frecuencia equilibrada.

o Eventos dedicados.

- Adaptación: Si durante la inspección se descubre que algún aspecto del proceso o del producto se está desviando de los límites aceptables, el equipo debe ajustar el rumbo inmediatamente.

o Acción correctiva rápida.

o Flexibilidad.

o Mejora continua.

Están basados en la experiencia del equipo en ese proceso, la cual se va ganando mediante ciclos cortos de retroalimentación.

Definido

Asume que al repetir el proceso, se obtiene el mismo resultados. La administración y control provienen de la predictibilidad del proceso definido.

Se puede lograr con cualquier tipo de ciclo de vida. Un ejemplo de proceso definido es el PUD.

leer manifiesto agil y guía de scrum

Ciclo de vida Vs Proceso

El ciclo de vida define el cuándo y el cómo mientras que el proceso define el qué.

Aparecen las 4 P:

El proceso, automatizado con herramientas, se adapta al proyecto, al cual se incorporan personas, y de este se obtiene como resultado un producto.

- Proceso definido: Secuencia de pasos
- Personas: Habilidades, entrenamiento, motivación.
- Proyecto
- Producto: Bienes o servicios.
- Herramientas: Equipos tecnológicos.

Componentes de Proyecto de Sistemas de Información

Proyecto

Unidad de gestión que organiza y coordina recursos y personas con el fin de desarrollar un producto o servicio. Se apoya en un proceso como guía.

- Entradas:
 - o Requerimientos
 - o Ideas
 - o Tiempo
- Proceso: Secuencia estructurada de pasos ejecutados con un propósito dado (IEEE)
- Salida: Producto o servicio

Debe cumplir con las siguientes características:

- Orientado a objetivos: Un proyecto está dirigido a obtener resultados especificados en los objetivos. Estos objetivos guían al proyecto, deben ser claros y alcanzables, no ambiguos.
- Duración Limitada: Es temporal. Cuando se alcanzan los objetivos, el proyecto termina.
- Tareas Interrelacionadas basadas en esfuerzos y recursos.
- Únicos: Todos los proyectos tienen características que los hacen únicos, por más similares que sean.

Alcance del producto: Todas las características incluidas (especificación de requerimientos)

Alcance del Proyecto: Todo el trabajo necesario para terminar el proyecto.

Administración de Proyectos

Aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto.

Incluye identificar requerimientos, establecer objetivos claros y alcanzables y adaptar las especificaciones, planes y enfoque a los diferentes intereses de los involucrados.

La gestión de proyectos es una disciplina que se apoya en dos subdisciplinas clave:

1. Planificación: Consiste en definir anticipadamente las actividades, recursos, tiempos y responsables necesarios para alcanzar los objetivos del proyecto.
2. Seguimiento y supervisión (Monitoreo y control)
 - a. Monitoreo: Se realiza de forma continua a lo largo del proyecto. Permite evaluar el avance y detectar posibles desvíos en tiempo real.
 - b. Control: Se enfoca en momentos específicos, llamados hitos, para verificar si se han alcanzado los objetivos parciales establecidos.

Tenemos que tener en cuenta la restricción triple (Triple Constrain) que afecta directamente a la calidad del proyecto:

- Tiempo: Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente.
- Costo: Define el costo de todos los recursos implicados en el desarrollo del proyecto.
- Objetivo del proyecto (alcance): Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente.

El costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto. En enfoques tradicionales los requerimientos se definen al comienzo del proyecto, por lo que cualquier modificación o cambio en el alcance del producto final, modifica el tiempo y presupuesto de este.

En enfoque ágil se trata de dejar el tiempo y los recursos fijos dejando que el alcance evolucione con el tiempo.

Plan de proyecto

Define qué, cuándo, cómo y quién.

1. Definición del objetivo. Debe estar alineado con a ERS.
2. Definición del Alcance del Proyecto: Tareas y actividades que se deben hacer para entregar el producto o servicio con todas las características y funciones especificadas. El cumplimiento del alcance se mide contra la ERS. Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto.
3. Definición de Proceso y Ciclo de Vida: Es importante tener en cuenta la información del contexto del desarrollo: objetivos, alcances, personas y recursos disponibles, clientes, forma de trabajo de personas, etc.
4. Definición del equipo del proyecto e involucrados externos.
5. Estimaciones: Se estima el tamaño, esfuerzo, calendario, costo y recursos críticos.
 - a. Basados en experiencias: Se basa en datos históricos y juicio experto.
 - b. Basados en Recursos: Análisis de los componentes del producto que se quieren construir.
 - c. Algorítmicos: fórmulas matemáticas.
6. Gestión de Riesgos: Posibles amenazas que puede tener nuestro proyecto. Tenemos en cuenta la exposición (impacto x probabilidad de ocurrencia)
Los riesgos deben ser gestionados: Identificar amenazas → Analizar impacto y probabilidad → Planificar estrategias de mitigación → Seguimiento con control constante → Aprendizaje
7. Asignación de Recursos.

8. Programación de Proyectos: En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo.
9. Definición de Controles: Toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido.
10. Definición de Métricas: Instrumentos de control. Las métricas básicas son tamaño, esfuerzo, tiempo calendario y defectos.
Se realizan en distintos niveles:
 - a. Organización
 - b. Equipo de Desarrollo
 - c. Desarrollador
 También se dividen en 3 dominios: Proceso (contexto de organización), proyecto (independiente p/ cada proyecto) y producto

Unidad 2: Gestión Lean-Ágil de Productos de Software

Filosofía Ágil y Manifiesto Ágil

Ágil es una ideología con un conjunto definido de principios que guían el desarrollo del producto, no es una metodología ni un proceso.

Los métodos ágiles son adaptables en lugar de predictivos y orientados a la gente en lugar de orientados al proceso.

Valores ágiles:

- Individuos e interacciones sobre procesos y herramientas.
- Software funcionando sobre documentación extensiva.
- Colaborar con el cliente sobre negociación contractual.
- Responder al cambio sobre seguir un plan.

Principios del Manifiesto:

1. Nuestra mayor prioridad es satisfacer al cliente
2. Aceptar que los requisitos cambien
3. Entregar software funcional frecuentemente
4. Los responsables de negocios, diseñadores y desarrolladores deben trabajar juntos día a día durante el proyecto
5. Desarrollamos proyectos en torno a individuos motivados
6. El método más eficiente de comunicar información es conversaciones cara a cara
7. El software funcionando es la principal medida de éxito
8. Los procesos ágiles promueven el desarrollo sostenible
9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad
10. La simplicidad es esencial
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados
12. A intervalos regulares el equipo reflexiona sobre como ser más efectivo y de acuerdo a esto ajustan su comportamiento

Al desarrollar en ágil, con el cliente se plantea como fijo los recursos y el tiempo (por iteración) y se estima el alcance, dirigido por el valor que recibe el cliente. En los equipos tradicionales, lo que queda fijo son los requisitos y se estiman los recursos y el tiempo dirigido por un plan.

Requerimientos ágiles

Se basa en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido.

Los requerimientos ágiles se basan en los siguientes pilares:

1. Usa el valor para construir el producto correcto.
2. Usa historias y modelos para mostrar qué construir.
3. Determina sólo aquello que es suficiente.

Product Backlog

Lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. Cada nuevo requerimiento se debe priorizar y agregar a la pila.

User Stories - US

Descripción corta, simple y centrada en el usuario de una funcionalidad o necesidad del sistema. Expresa una intención. Son consideradas un ítem de configuración de producto y pueden descartarse una vez que se hayan finalizado. Son escritas por el PO, quien las prioriza en el Product Backlog para que aquellas con mayor prioridad se modelen con mayor detalle y se estimen. En cada iteración se implementan la US con mayor prioridad. Están conformadas por las tres C:

1. Tarjeta: se expresa con una frase verbal de la siguiente manera:
Como <nombre del rol> yo puedo <actividad> de forma tal que <valor de negocio que recibo>.
Además se incluye:
 - a. Criterios de aceptación: nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación. Está escrito en términos objetivos y concretos. Definen intención del usuario, no funcionalidad. Son independientes de la implementación y se describen a alto nivel.
 - Definen límites y alcance.
 - Ayudan a que el equipo tenga una visión compartida de la US.
 - Ayudan a desarrolladores a y testers a derivar pruebas.
 - Ayudan a desarrolladores a saber cuándo parar de agregar funcionalidades.
 - b. Pruebas de aceptación: Declaraciones de intención de que hay que probar. Se prueban escenarios exitosos y escenarios que fallen.
2. Conversación: Se espera que la historia genere conversaciones entre el equipo, el Product Owner y los stakeholders. La conversación se materializa de forma distribuida en la tarjeta y en las pruebas de aceptación.
3. Confirmación: Hace referencia a establecer criterios de aceptación. Define un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó.

Una buena US debe seguir el modelo *INVEST*:

- Independent: Implementable en cualquier orden.
- Negotiable: Describen el "qué", no el "cómo".
- Valuable: Tienen valor para el cliente.
- Estimable: Ayuda al cliente a armar un ranking en base a costos.
- Small: Realizable en una sola iteración.
- Testable: Demostrar que fue implementada.

Definición de Listo (Definition of Ready - DoR): Nos dice cuando una US está lista para ser implementada.

- Valor de negocio claramente expresado.
- Detalles suficientemente comprendidos.
- Dependencias identificadas y no hay dependencias externas que puedan impedir que se complete el ítem del Product Backlog (PBI).
- PBI estimado y es lo suficientemente pequeño para ser completado en un sprint.
- Criterios de aceptación claros y testeables.
- Criterios de performance (si hay), claros y testeables.

Definición de Hecho (Definition of Done - DoD): Nos dice cuando una US está terminada y lista para ser desplegada en producción.

- Diseño revisado
- Código completo
- Documentación actualizada
- Probado
- Cero defectos conocidos
- Pruebas de aceptación realizada
- En servidores de producción

Spikes

Son un tipo especial de US utilizado para quitar riesgo e incertidumbre de una US. Pueden ser técnicas o funcionales. Las Spikes técnicas se usan para investigar enfoques técnicos en el dominio de la solución en situaciones en las que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo. Las funcionales se usan cuando hay incertidumbre de cómo el usuario interactuará con el sistema. Son evaluadas con prototipos para obtener retroalimentación de los usuarios o involucrados. Los Spikes deben ser demostrables, estimables y aceptables. Deben reservarse para incógnitas grandes como última opción. En general, el spike se implementa en una iteración separada de la historia resultante.

Estimaciones Ágiles

Las estimaciones son predicciones numéricas asociadas a una probabilidad de que cierto evento ocurra. No se deben tomar como un compromiso, pero sí puede ser una gran respuesta sobre si el trabajo planificado es factible de realizar y ser una gran protección para el equipo.

En un proyecto se estima en el siguiente orden:

1. Tamaño: cantidad de trabajo necesaria para lograr el objetivo. Para hacer esta medición, se debe tomar un metro patrón que será elegido para realizarla (story point). Define el QUÉ.
2. Esfuerzo: determina cuántas horas personas lineales se necesitan para completarlo. Define el CÓMO.
3. Tiempo: Distribuye las horas lineales en el equipo. Define el CUÁNDO.
4. Costo: se deriva del esfuerzo. Define el CUÁNTO.

Axioma Pragmático: No ser dogmático sobre nada. Ser pragmático es la clave en casi todo.

El pragmatismo: La aceptación de la incertidumbre empírica, utilizando marcos de trabajo iterativos para adaptarnos a la realidad cambiante.

La estimación en tiempo depende de las habilidades individuales de los desarrolladores, conocimientos técnicos específicos, sesgos, familiaridad con el dominio del negocio, experiencia en la industria.

En el paradigma ágil se estiman US utilizando como medida de tamaño los Story Points, que es una medida de estimación relativa no basada en el tiempo.

La estimación relativa funciona bien porque las personas son buenas comparando cosas. Se obtiene una mejor dinámica grupal.

Story Points - SP

Mide el *tamaño* de una US en términos de la cantidad de trabajo necesaria para producirla. Al ser relativa, es propia de cada equipo de trabajo y tiene en cuenta los siguientes aspectos:

1. Complejidad: Magnitud, esfuerzo mental y complejidad arquitectónica.
2. Esfuerzo: Cantidad de horas que pensamos que nos lleva.
3. Incertidumbre: Nivel de incertidumbre que tengo sobre el requerimiento.

El crecimiento de la complejidad, esfuerzo e incertidumbre crece de forma exponencial. Para hacer estimaciones usamos serie de Fibonacci.

Velocidad/velocity

Medida del progreso de un equipo. Se calcula sumando el número de SP que el equipo realmente completó totalmente durante la iteración. La Velocidad corrige los errores de estimación. Si la US se completó parcialmente, no se cuentan sus SP.

El promedio de velocidades a lo largo de sucesivas iteraciones permite al equipo a estimar mejor.

Capacidad

Horas de trabajo disponibles por día * días de la iteración.

Poker Planning

Es un método de estimación. Todo el equipo que va a trabajar en la historia de usuario va a participar de la estimación. El equipo estima su propio trabajo.

Pasos:

0. Prerrequisitos
 - a. Listado de US y cartas con los números.
 - b. Determinar US Canónica que se usa como base de comparación. Suele tener valor 1.
1. La US a ser estimada se lee a todo el equipo.
2. Los estimadores discuten la US.
3. Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
4. Todas las cartas se exponen al mismo tiempo.
5. Si todos los estimadores seleccionan el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Después de la charla, GOTO to 3.
6. Se toma la próxima story, GOTO 2.

Diferencias con las estimaciones en ambientes tradicionales

Dimensiones

En Agile se tiene fijo el tiempo y los recursos afectados para trabajar, dejando variable el alcance. En los ambientes tradicionales, el alcance es lo que está fijo y el producto se encuentra definido. Por esta razón en los procesos definidos se

estima primero el tamaño, ya que es la base para estimar el resto.

Precisión

Las estimaciones en los ambientes tradicionales buscan ser más precisas, debido a que son las bases para luego planificar.

Modo

Las estimaciones ágiles son relativas, a diferencia de las estimaciones en ambientes tradicionales que son absolutas.

Responsable

En tradicional, las estimaciones las realiza el líder del proyecto, por lo que no es representativa de la cantidad de trabajo que requiere una persona del equipo para terminar una funcionalidad del producto. En los ambientes ágiles es el equipo el que realiza la estimación.

Relación con Plaificación

En ambientes tradicionales, muchas veces se asume las estimaciones como compromisos. En agile no implican un compromiso. Una falla en la estimación no implica extender el tiempo, lo que se ajusta es el alcance.

Momento

En el enfoque tradicional se estima al comienzo del proyecto, luego de la especificación de requerimientos y luego del diseño. Las estimaciones a lo largo del proyecto se van haciendo cada vez más certeras. En ágiles se estima al comienzo de cada sprint.

Gestión de Productos

El ciclo de vida de un producto está compuesto por:

1. Fase de introducción
2. Fase de crecimiento
3. Fase de madurez
4. Fase de declive

En general, muchas de las características de un producto no se usan nunca o se usan muy poco, por lo que es importante descubrir qué es lo que le da valor al usuario y focalizarse en eso.

La idea es, antes de empezar a construir un producto, validar la hipótesis de que ese producto le agrega valor al cliente.

El mayor riesgo no es fallar técnicamente, es construir algo que el cliente no necesita. Lo que buscamos es agregar valor al producto de software.

El dinosaurio Lean/Agile

Usamos la metodología Lean Startup.

Al crear software se debe iniciar con una hipótesis de valor único o *Propuesta de Valor Única* (UVP). Cada producto o servicio es único. La idea es invertir el menor esfuerzo posible para validar la hipótesis.

El siguiente paso es crear un *Producto Mínimo Viable* (MVP) para probar esta hipótesis. Este se centra en la propuesta de valor. Si cada cliente ve un potencial diferente para el MVP, aún no hay un mercado para el producto; por el contrario, si todas las respuestas apuntan en el mismo sentido (convergen), entonces tiene sentido revisar la hipótesis pivotando hacia un MVP2 basado en el aprendizaje reciente.

En función del feedback y la investigación, se encuentran áreas de posible crecimiento para un producto.

En caso de encontrar áreas con fuerte indicación de valor, se define una *Característica Mínima Comercializable* (MMF).

Esta tiene el objetivo de aportar valor y supone una certeza de que existe valor en esa área.

Si tenemos incertidumbre, se plantea una hipótesis centrada en una característica en vez de un producto y se genera una *Característica Mínima Viable* (MVF).

- **Minimum Viable Product - MVP:** Versión de nuevo producto creado con el menor esfuerzo posible. Dirigido a clientes potenciales con el objetivo de obtener aprendizaje validado. No está destinado a producción, se ve como el producto terminado. Tiene que tener el valor suficiente para que las personas est[en] dispuestas a usarlo o comprarlo inicialmente.
- **Minimum Marketable Feature - MMF:** Pieza más pequeña de funcionalidad que puede ser liberada. Es una funcionalidad comerciable de valor para los usuarios y la organización. Es parte de un MMR o MMP.
- **Minimum Marketable Product - MMP:** Primer release de un MMR dirigido a los primeros usuarios. Es un producto funcional con foco en estabilidad técnica y lanzamiento al mercado.
- **Minimum Marketable Release - MMR:** Release de un producto que tiene el conjunto de características más pequeño posible. Le ofrece valor nuevo a los usuarios y satisface sus necesidades actuales.

El MVP debe abarcar todos los aspectos del producto:

Funcionalidad: Tiene todas las funciones necesarias para solucionar un problema específico.

Confiabilidad: Involucra early adopters.

Usabilidad: Resulta útil al público objetivo.

Diseño: Satisface el aspecto visual y de interacción.

Matriz de Priorización:

	Urgente	No Urgente
-----	-----	-----
Importante	Hacer	Planear
No Importante	Delegar	Eliminar

Para las organizaciones modernas, cruzar de la viabilidad técnica al amor del usuario es la grieta más difícil de sortear.

Minimum Lovable Product (MLP): Logra que los usuarios elijan mi producto frente a otros similares. Foco en UI avanzada, impacto emocional y reducción de fricción del usuario.

Fomenta la lealtad y adopción temprana.

Para lograr esto debemos entender y conocer el público objetivo -> definir propuesta que aporte valor -> resolver al menos una de las necesidades del usuario de manera excelente -> sorprender al usuario.

Scrum

Es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

- Responsabilidades
 - Scrum Master
 - Product Owner
 - Developers
- Reuniones
 - Sprint Planning
 - Daily Scrum
 - Sprint Review
 - Sprint Retrospective
 - Refinamiento del PB
- Artefactos
 - Product Backlog
 - Sprint Backlog
 - Versión del Producto

Niveles de planificación

- Día: Foco en como completar lo comprometido. Por equipo Scrum.
- Iteración: Qué aspectos entregar en el siguiente sprint. Equipo Scrum
- Release: Balancear valor de cliente y calidad global con restricciones de alcance, cronograma y presupuesto. Equipo Scrum y Stakeholders.
- Producto: Visión y evolución del producto a través del tiempo. PO y Stakeholders.
- Portfolio: Administración de Portfolio de producto. PO y Stakeholders.

Unidad 3: Gestión del Software como Producto

Gestión de Configuración de Software - SCM

Software Configuration Management

Software

El Software es el conjunto de programas, procedimientos, reglas, documentación y datos. Es información estructurada con propiedades lógicas y funcionales.

El software cambia con el tiempo. El desafío es mantener la integridad del producto a medida que surgen los cambios.

La gestión de configuración tiene el propósito de mantener la integridad del producto.

Es Software cambia por:

- Cambios del negocio y requerimientos.
- Soporte de cambios de productos asociados.
- Reorganización de las prioridades de la empresa por crecimiento.
- Cambios en el presupuesto.
- Defectos encontrados a corregir.
- Oportunidades de mejora.

SCM

Disciplina de soporte transversal a todo el proyecto, relevante para el producto a lo largo de su ciclo de vida.

Las disciplinas de soporte son:

- Control de Calidad de Proceso
- Control de Calidad de Producto
- Prueba de software

Definición: Disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos.

Sus actividades principales son:

1. Control de Cambios.
2. Identificación de Ítems de Configuración.
3. Auditorías de Configuración de Software.
4. Informes de Estado.

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. para esto debe identificar la configuración, controlar sus cambios sistemáticamente y mantener su integridad y origen.

Integridad del producto:

- Satisface necesidades del usuario.
- Puede ser fácil y completamente rastreado durante su ciclo de vida.
- Satisface criterios de performance.
- Cumple con sus expectativas de costo.

Conceptos clave:

- Ítem de configuración (IC): Todos y cada uno de los artefactos que forman parte del producto o proyecto y que puede sufrir cambios o necesita ser compartido entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución. Definido por el equipo del proyecto. Pueden ser de proyecto (de iteración o de todo el proyecto) o del producto.
- Versión: Forma particular de un artefacto en un instante o contexto dado. El control de versiones se refiere a la evolución de un único IC o de cada IC por separado.
- Variante: Versión de un IC que evoluciona por separado. Representan configuraciones alternativas.
- Configuración del Software: Conjunto de IC con su correspondiente versión en un momento determinado.
- Repositorio: Responsable de almacenar y mantener la historia de cada IC con sus atributos y relaciones. Pueden ser:
 - Centralizado: Un servidor contiene todos los archivos con sus versiones. Los administradores tienen mayor control.
 - Descentralizado: Cada cliente tiene una copia exactamente igual del repositorio completo.
- Línea Base: Configuración revisada formalmente sobre la que se ha llegado a un acuerdo. Sirve como base para desarrollos posteriores. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto. Puede ser:
 - De especificación: Requerimientos, diseño.
 - De Productos: Producto que ha pasado por un control de calidad definido previamente.
- Ramas: Sirven para bifurcar el desarrollo. Permiten la experimentación, pueden ser descartadas. Cuando se integra una rama, sus cambios se llevan a la rama principal por medio de un 'merge'.

Actividades Fundamentales de la SCM

Identificación de IC:

Identificación unívoca de cada IC, convenciones y reglas de nombrado, definición de la Estructura del Repositorio, ubicación dentro de la estructura del repositorio. Se debe tener en cuenta al momento de identificar los ítems, la duración de su integridad.

Pueden ser:

- Ítems de producto: Tienen el ciclo de vida más largo, y se mantienen mientras el producto exista.
- Ítems de proyecto: Son aquellos en que el ciclo de vida dura lo mismo que el proyecto.
- Ítems de iteración: Su ciclo de vida dura una única iteración.

Control de Cambios:

Surge de un requerimiento de cambio a uno o varios IC que se encuentran en una línea base. Es un procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio.

1. Se recibe una propuesta de cambio sobre una línea base determinada, no sobre un ítem.

2. El comité de control de cambios realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, los efectos secundarios y el impacto global sobre la funcionalidad y la arquitectura del producto.
3. En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios para revisar y auditar.
4. Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente, es decir que realiza una revisión de partes.
5. Finalmente se notifica a los interesados los cambios realizados sobre la línea base.

El CCC está formado por representantes de todas las áreas involucradas en el desarrollo: Análisis, Diseño, Implementación, Testing y otros interesados.

Auditorías de Gestión de Configuración de Software:

Auditoría es una validación externa al proyecto que asegura que cumpla con lo que tengo definido.

- Física (PCA): Asegura que lo que está indicado para cada ICS en la línea base o la actualización se ha alcanzado realmente. Evalúa la estructura de lo definido en el plan.
- Funcional (FCA): Evaluación independiente del producto de software. Controla que la funcionalidad y performance real de cada IC sea consistente con la especificación de requerimientos.

Usa dos procesos básicos:

- Validación: El problema es resuelto de manera apropiado y el usuario obtiene el producto correcto (FCA).
- Verificación: Asegura que el producto cumple con los objetivos preestablecidos, definidos en la documentación de línea base. Las funciones son llevadas a cabo con éxito y los test cases correctos. Se fija que el producto es resuelto de manera apropiada (PCA).

Registro e Informe de Estado:

Mantiene registros de la evolución del sistema.

Maneja mucha información y salida por lo que se suele implementar dentro de procesos automáticos.

Ayuda al responsable de la SCM a llevar a delante la gestión. Ver cambios aprobados y rechazados, el estado de un IC, versión que implementa un cambio, etc...

Plan de Gestión de Configuración

Es parte del plan de desarrollo de software. Documenta todo el detalle de las definiciones.

Debe incluir las definiciones de los IC, herramientas a utilizar, roles e integrantes del comité de control de cambio, procedimiento formal de cambios y plantillas de auditorías.

CSM en ambientes Ágiles

La gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores. Va de la mano con el manifiesto ágil porque recordemos que se buscaba estar siempre preparados para el cambio y justamente, la gestión de configuración busca responder a los cambios y mantener la integridad del producto. El manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto.

El comité de control de cambio no existe, porque se le da la responsabilidad a todo el equipo, no tiene sentido establecer roles específicos.

